

Exploring the Application of IPOPT on Selected Hock-Schittkowski Test Problems from the CUTEst Dataset

Alex Ojemann

Abstract

This report explores the implementation and testing of the IPOPT algorithm on selected HS problems from the CUTEst dataset. The work aims to demonstrate the robustness and utility of IPOPT by testing it on problems not covered in standard examples. Challenges encountered during the process are discussed, alongside comparisons with the barrier method.

1 Motivation

IPOPT, or the Interior Point OPTimizer, is a widely-used optimization algorithm for solving large-scale nonlinear problems with constraints. Its ability to handle problems with nonlinear objectives and constraints, while maintaining efficiency and scalability, makes it a valuable tool in optimization.

The CUTEst dataset provides a standardized collection of optimization problems, making it a benchmark for testing algorithms. Among the dataset, the HS problems (Hock-Schittkowski test problems) stand out due to their clear and accessible documentation, offering a range of problem types with varying complexities. These problems were selected for their relatively quick solving times and well-documented solutions, making them ideal for focused testing.

In this work, I focus on testing IPOPT on HS problems that were not included in the standard cypopt examples. The selection includes problems with quadratic and nonlinear constraints, providing a diverse testbed to validate IPOPT's performance.

2 Literature Survey

The development of IPOPT is rooted in a rich history of optimization algorithms. Below is an overview of key contributions:

2.1 Simplex Method

Dantzig's simplex method laid the foundation for linear programming, introducing systematic approaches to traverse feasible regions efficiently [1]. The algorithm's success lies in its ability to move along the edges of the feasible region to find the optimal vertex. However, simplex is limited to linear programming and struggles with nonlinearity and scalability.

2.2 Early Penalty Methods

Penalty methods introduced penalties for constraint violations directly into the objective function [3]. These methods allowed unconstrained optimization techniques to be applied but suffered from ill-conditioning as the penalty parameter increased. This limitation motivated the development of barrier and interior-point methods.

2.3 Barrier Methods

Barrier methods use logarithmic terms to enforce constraints [3]. The primary challenge was numerical instability near constraint boundaries. Although effective for small problems, these methods struggled with large-scale or highly nonlinear constraints.

2.4 Sequential Quadratic Programming (SQP)

SQP methods iteratively solve quadratic subproblems to approximate the original optimization problem [7]. These methods improved the handling of nonlinear constraints but faced challenges with large-scale problems and complex constraint structures.

2.5 Primal-Dual Interior Point Methods

Karmarkar's work extended linear programming techniques to nonlinear problems [6]. Primal-dual methods, which solve the Karush-Kuhn-Tucker (KKT) conditions directly, improved robustness and scalability. Wächter and Biegler refined these methods, incorporating adaptive strategies and filter line searches to improve performance [8].

2.6 Filter Methods

Filter methods replaced penalty terms with a multi-criteria approach, balancing feasibility and optimality [4]. This innovation addressed issues of ill-conditioning and facilitated global convergence, forming a critical component of IPOPT.

2.7 How IPOPT Builds on Previous Algorithms

IPOPT [8] synthesizes the strengths of its predecessors while addressing their limitations. It combines primal-dual interior point strategies with adaptive barrier methods, leveraging the scalability of Karmarkar's techniques and the robustness of filter methods. The adaptive handling of barrier parameters ensures numerical stability near constraint boundaries, a challenge for traditional barrier methods. Moreover, IPOPT's efficient implementation of SQP-like updates enables it to tackle nonlinear constraints effectively. By integrating filter methods, IPOPT circumvents the need for penalty terms, enhancing global convergence and computational efficiency. These innovations collectively make IPOPT a powerful tool for solving complex, large-scale optimization problems.

The progression from simplex to primal-dual methods highlights advancements in handling nonlinearity and large-scale problems, culminating in robust solvers like IPOPT.

3 HS Problems Description

The selected HS problems include:

- **HS25:** A nonlinear objective with a single equality constraint.
- **HS38:** Features a nonlinear objective and multiple nonlinear constraints.
- **HS52:** A high-dimensional problem with linear constraints.
- **HS80:** A complex problem with quadratic and cubic constraints.

These problems were chosen for their diversity and relative efficiency in solving, allowing for detailed analysis without excessive computational expense. Their well-documented solutions provided a reliable benchmark for evaluating IPOPT's performance.

4 Challenges and Implementation Details

4.1 Implementation Difficulties

Implementing IPOPT for the HS problems presented several challenges. Since IPOPT required a Linux environment, I used the Windows Subsystem for Linux (WSL). Setting up WSL and navigating a Linux-based system posed difficulties for someone unfamiliar with this operating system. Issues with package compatibility and system dependencies further complicated the setup process.

4.2 Constraint Scaling Method

Another significant challenge was tuning the constraint scaling method. IPOPT's performance depends heavily on proper scaling of constraints to avoid numerical instability. For example, poorly scaled constraints can lead to infeasible solutions or slow convergence. To address this, I experimented with various scaling parameters to ensure that all constraints were appropriately normalized. This iterative tuning process was essential for achieving accurate and reliable results.

5 General Solution

IPOPT solves nonlinear optimization problems by combining barrier methods with primal-dual interior-point strategies. Its adaptive handling of constraints and efficient search directions make it a robust solver. Adapting the HS71 example, I extended the framework to handle additional HS problems by defining their specific constraints, Jacobians, and Hessians.

6 Technical Description of IPOPT

IPOPT iteratively solves the following problem:

$$\begin{aligned} \min_{x \in \mathbb{R}^n} \quad & f(x) \\ \text{s.t.} \quad & g_L \leq g(x) \leq g_U, \\ & x_L \leq x \leq x_U. \end{aligned}$$

6.1 Karush-Kuhn-Tucker (KKT) Conditions

The KKT conditions provide the necessary optimality criteria for constrained optimization problems. For IPOPT, the conditions can be expressed as:

$$\begin{aligned} \nabla f(x) + \sum_{i=1}^m \lambda_i \nabla g_i(x) &= 0, \\ g_L &\leq g(x) \leq g_U, \quad x_L \leq x \leq x_U, \\ \lambda_i &\geq 0 \text{ for inequality constraints,} \\ \lambda_i(g_i(x) - g_U) &= 0, \quad \lambda_i(g_L - g_i(x)) = 0. \end{aligned}$$

Here, λ_i are the Lagrange multipliers, $g(x)$ represents the constraints, and x denotes the decision variables.

6.2 Algorithm Steps

Key steps of the IPOPT algorithm include:

1. **Barrier Problem Formulation:** IPOPT augments the objective function with barrier terms for inequality constraints:

$$\phi(x, \mu) = f(x) - \mu \sum \ln(s_i),$$

where s_i are slack variables and μ is the barrier parameter.

2. **Newton's Method:** The KKT conditions are solved using a Newton-type method to determine the search direction. The linear system includes both primal and dual variables.
3. **Filter Line Search:** Instead of penalty terms, IPOPT employs a filter to balance feasibility and optimality, ensuring global convergence.
4. **Barrier Parameter Update:** μ is reduced adaptively, refining the solution as the algorithm progresses.

7 Numerical Results

7.1 HS71 Example

HS71 serves as the canonical example included in cyipopt [2], which is the Python package I used to generate solutions. I reproduced the results for validation before extending the methodology to other HS problems. I retrieved all of these HS problems from the CUTEEST dataset [5]. The results for HS71 are shown below:

Problem	Initial Objective	Final Objective	Initial Feasibility	Final Feasibility	Convergence Status	Iterations
HS71	16	17.014	No	Yes	Converged	8

Table 1: HS71 Results Using IPOPT

7.2 Additional HS Examples

The table below summarizes the performance of IPOPT on the selected HS problems other than HS71:

Problem	Initial Objective	Final Objective	Initial Feasibility	Final Feasibility	Convergence Status	Iterations
HS25	32.835	1.819e-16	Yes	Yes	Converged	27
HS38	19192	0	Yes	Yes	Converged	39
HS52	42	5.327	No	Yes	Converged	90
HS80	3.355e-4	0.0539	No	Yes	Converged	270

Table 2: IPOPT Results on Selected HS Problems

It’s important to note that there are examples in which the objective function is lower for the initial value of x than the final value, but this is only occurs when the initial value of x is infeasible.

7.3 Barrier Method Comparison

All attempts to solve the same problems using early barrier methods failed to converge. This was due to:

- Infeasible starting points.
- Sensitivity to ill-conditioning near constraint boundaries.
- Poor handling of nonlinear constraints.

8 Discussion and Conclusion

The results demonstrate the robustness of IPOPT in solving complex nonlinear problems. Unlike the barrier method, IPOPT effectively balances feasibility and optimality, handling nonlinearities and bounds efficiently.

8.1 Why Barrier Methods Failed

Barrier methods struggled due to their sensitivity to infeasible starting points and ill-conditioning near constraint boundaries. Their reliance on penalty terms often led to numerical instability, especially for problems with nonlinear constraints or large condition numbers.

8.2 Future Directions

Possible future directions include:

- **Comparison with Advanced Solvers:** Testing IPOPT against solvers like SNOPT and KNITRO will highlight its relative strengths and weaknesses more so than Barrier Methods which we know are not well suited to these problems.
- **Exploring Additional Problems:** Extending the analysis to more problems in the CUTEst dataset will provide broader validation. There are 1539 total problems in the CUTEst dataset, and this paper only explores a few of them, but many of these problems will take much longer for the algorithm to solve.
- **Real-World Applications:** Applying IPOPT to industry problems will demonstrate its practical utility and scalability.

Each direction promises to enhance our understanding of IPOPT’s capabilities and limitations, driving further advancements in optimization.

References

- [1] George B. Dantzig. *Linear Programming and Extensions*. Princeton University Press, 1951.
- [2] CyIopt Developers. Cyipopt: A python wrapper for ipopt, 2024. Available at <https://github.com/matthias-k/cyipopt>.
- [3] Anthony V. Fiacco and Garth P. McCormick. Nonlinear programming: Sequential unconstrained minimization techniques. *SIAM Journal on Applied Mathematics*, 15:1–15, 1968.
- [4] Roger Fletcher and Sven Leyffer. Nonlinear programming without a penalty function. *Mathematical Programming*, 91:239–269, 2002.
- [5] Jaroslav Fowkes and Lindon Roberts. Pycutest: A python interface to the cutest optimization test environment, 2022. Available at <https://github.com/jfowkes/pycutest>.
- [6] Narendra Karmarkar. A new polynomial-time algorithm for linear programming. *Combinatorica*, 4(4):373–395, 1984.
- [7] M.J.D. Powell. A fast algorithm for nonlinear programming calculations. *Numerical Analysis*, 630:144–157, 1978.
- [8] Andreas Wächter and Lorenz T. Biegler. On the implementation of a primal-dual interior point filter line search algorithm for large-scale nonlinear programming. *Mathematical Programming*, 106(1):25–57, 2006.